
PalmSource™ 99
Writing Applications with the GCC Tool Chain
John Marshall
Toolsmith
3Com
More connected.

A Little GCC History

PalmSource 99

- 1984: The GNU project, and later the Free Software Foundation and GCC are born
- January 1997: GCC 2.7.2.2 released
- July 1997: prc-tools 0.5.0 released, containing
  - GCC 2.7.2.2
  - GNU linker and debugger of similar vintage
  - build-prc, a custom post-linker
  - header files for Palm OS® software version 2.0
- August 1999: GCC 2.95.1 released
- October 1999: prc-tools updated

Major New prc-tools Features

PalmSource 99

- Multiple code resources
- Access to all Palm OS® system API functions
- Some C++ support
- Support for building shared libraries
- Control over callbacks and the globals register in shared libraries
- 2½ years of core GCC compiler development

FAQs We Won't Hear Anymore

PalmSource 99

- "relocation truncated to fit: DISP16 foo"
- Why does my app crash in callbacks?
  or
  What's this CALLBACK_PROLOGUE stuff?
- "region coderes is full"
  - Actually, we'll still get this message, but now there'll be a solution
- Why don't my objects' constructors get called?
- What's wrong with the line numbers in gdb?

Why Choose GCC?

PalmSource 99

- It doesn't cost very much
- It's free
- Good error reporting and standards compliance
- The optimiser is correct
  - Very few problems due to turning optimisation on
- Speed and size of optimised code?
  - I think the jury is still out on that one

Contents of a Tool Chain for the Palm OS® Platform

PalmSource 99

- Something to compile the source code
  gcc, as, ld
- Something for resources
  pilrc
- A “post-linker” to tie the two together
  build-prc
- The Palm OS SDK
  - Headers
  - Libraries
- Not strictly necessary, but very nice to have
  - Poser, a debugger, code examples, an IDE

Hello World

PalmSource 99

```
#include <Pilot.h>

UInt32 PilotMain(UInt16 cmd, void *cmdPBP,
                 UInt16 launchFlags) {
   if (cmd==sysAppLaunchCmdNormalLaunch) {
      EventType e;
      WinDrawChars (“Hello world”, 11,
                    50, 50);
      do {
         EvtGetEvent (&e, evtWaitForever);
         SysHandleEvent (&e);
         } while (e.eType != appStopEvent);
      }
   return 0;
   }
```

Compiling Hello World

PalmSource 99

```
$ m68k-palmos-gcc -o hello hello.c
$ build-prc hello.prc “Hello” Helo hello
$
```

```
CC = m68k-palmos-gcc

hello.prc: hello
   build-prc hello.prc “Hello” Helo hello

hello: hello.o

hello.o: hello.c
```

Post-linker Improvements

- Options for specifying database header flags
- Incorporation of obj-res functionality
- Ability to read resources from .prc files
- Optionally controlled by a “database definition file” (.def) which can specify
  - Database header flags
  - Information needed for multiple code resources
  - Export lists for shared libraries
  - Other miscellaneous data which may arise

Memo Pad .def File

Old-style build-prc without .def file:

```
$ obj-res MemoCode
$ build-prc Memo.prc “Memo Pad” memo *.grc
```

.def file and new-style build-prc:

```
application {
   “Memo Pad” “memo”
   }

$ build-prc Memo.def MemoCode
```

Multiple Code Resources

PalmSource 99

- Add section attributes to functions to be placed in other sections
- Write a little assembly code to tell the startup code how many code resources need to be initialised
- But make sure that any functions used from globals-less launch codes stay in the main section!

Address Book Section Attributes

PalmSource 99

```
#define EDITFUNCS_SECTION \
   __attribute__ ((section “editfuncs”))

Boolean DetermineRecordName ( … )
   EDITFUNCS_SECTION;

void Lookup (char *s) EDITFUNCS_SECTION;
```

Address Book .def File and Makefile

PalmSource 99

```
application { “Address Book” “addr” }
multiple code { “editfuncs” “misc” }


Address.prc: Address.def Address rsrc…
     build-prc Address.def Address rsrc…

Address: Address.o AddressLookup.o \
         AddressDB.o… Address-sections.o

Address-sections.o: Address-sections.s

Address-sections.s: Address.def
     multigen Address.def
```

Shared Libraries

PalmSource 99

- Both “system libraries” and “GLibs”
- For system libraries
  - Entry point functions must take a RefNum as their first parameter
  - You must write an assembly language jump table to jump to the functions proper
- For GLibs
  - You need to write a different sort of assembly language jump table

Net Library .def File

PalmSource 99

```
syslib {
   “Net Library” “netl”
   }

export {
   NetLibOpen
   NetLibClose
   NetLibSleep
   NetLibWake
   NetLibSocketOpen
   NetLibSocketClose
   NetLibSocketOptionSet
   …
   }
```

Net Library Makefile

PalmSource 99

```
CC = m68k-palmos-gcc

NetLib.prc: NetLib.def NetLib rsrc…
     build-prc NetLib.def NetLib rsrc…

OBJS = NetLib-jumps.o Network.o Socket.o…

NetLib: $(OBJS)
     $(CC) $(OBJS) -nostartfiles

NetLib-jumps.o: NetLib-jumps.s

NetLib-jumps.s: NetLib.def
     stubgen NetLib.def
```

Great! Where Do I Get It?

PalmSource 99

- Look under “Tools” on devzone

  http://www.palm.com/devzone/

- Source code for the post-linker, and the patches to the GNU packages
- Binary distributions for
  - Linux
  - Windows

The TO DO List

- Submit the GCC, binutils, gdb patches back to the main FSF distributions
- Move from producing COFF files to ELF
  - Automatic stripping of unused functions
  - Slight optimisation of short jumps
  - Multiple code resources without attributes
- C++ support
  - Virtual functions accessible without globals
  - Exceptions
  - The standard library


PalmSource™ 99
3Com
More connected.

SwitchHack .def File

PalmSource 99

```
hack {
   “SwitchHack” “HACK”
   copy-prevention
   }

trap {
   SysUIAppSwitch {appswitch}
   0xA121 {keystroke}
   }
```

Floating Point Support

PalmSource 99

- Two potential problems:
  - Calling the systraps
  - Floating point formats
- Which floating point library functions?
  - OldFloatMgr?
  - NewFloatMgr?
  - Native GCC operators?
- How do I write a function to convert a string to a float? And vice versa?